

W2 Algorithmic Language

Thursday, October 1, 2026

READINGS Chomsky, *Syntactic Structures* (1957), excerpts

CONTENTS	1	Language and grammar	1
	2	Proto-theories of grammar	2
	3	Competence and performance	3
	4	Finite state grammars	4
	5	Limitations of finite state grammars	5
	6	Phrase structure grammars	6
	7	Limitations of phrase structure grammars	9
	8	The Chomsky hierarchy and computing machines	10
	9	Grammars as theories	10
		References	11

1 Language and grammar

Why *Syntactic Structures* hit so hard.

- **Grammar as machine.** SS took the apparently open-ended, “soft” problem of language production, and thought of it in mechanical terms, treating the human as a machine.
- **Method.** SS developed a series of simple but precise models that approximated English. Though the simpler models could not be correct, the problems with them were clearly pin-pointed. You could see how to keep going with the research project.
 - “pushing a precise but inadequate formulation to an unacceptable conclusion” (Preface, p. 5)
- **Level of Abstraction.** SS developed a way of stating the problem of grammar that was quite abstract, so that you could study the grammatical mechanism without looking inside the brain.

- **Generating the set.** The problem of a grammar is defined as generating the full set of sentences, the way the periodic table generates the full set of elements. It is not the psychological process of comprehension or of production. *This move to a “neutral” task is both controversial and what makes all of this possible.*

Defining language, defining grammar

- **Syntax.** “Syntax is the study of the principles and processes by which sentences are constructed” (p. 11).
- **Language.** A language is a set (finite or infinite) of sentences— intuitively, the set of all grammatical sentences in the language.
- **Grammar.** The grammar of a language “generates all of the grammatical sequences of L and none of the ungrammatical ones” (p. 13).
- **Grammatical.** A consciously accessible property of sentences that are properly formed within a language (though the rules of proper formation may not be consciously accessible.)

2 Proto-theories of grammar

- **Grammar as an infinite set of sentences.**
 - “One requirement that a grammar must certainly meet is that it be finite.” (p. 18)
 - Where does this requirement come from? Presumably: the finiteness of the brain.
 - If a grammar is finite, and languages are infinite, a grammar cannot be a list of sentences in the language.
- **Grammar as a corpus.**
 - “the set of grammatical sentences cannot be identified with any particular corpus of utterances” (p. 15)
- **Grammar as a statistical summary.**
 - “there appears to be no particular relation between order of approximation and grammaticalness” (p. 17)
 - Neither (1) nor (2) have ever been used (as of the writing of SS), but only (1) is grammatical.

- (1) Colorless green ideas sleep furiously.
- (2) Furiously sleep ideas green colorless.

- **Grammar as semantics.**

- “the notion ‘grammatical’ cannot be identified with ‘meaningful’ or ‘significant’ in any semantic sense” (p. 15)
- Sentences (3) and (4) are both meaningful and grammatical. Sentences (5) and (6) are meaningful but not grammatical.
- “grammar is autonomous and independent of meaning” (p. 17)

- (3) have you a book on modern music?
- (4) the book seems interesting.
- (5) read you a book on modern music?
- (6) the child seems sleeping. (p. 15)

3 Competence and performance

- **Infinite language?** Does it make sense to say that we can know a language, if languages are infinite sets of sentences?
 - **Problem 1:** we can only store finite information in our brains.
 - **Problem 2:** we can only utter/hear finite number of sentences in our lifetimes.
- **Competence and performance.** Chomsky later drew the distinct between **performance** (what behavior an organism actually carries out) and **competence** (its underlying ability). See Chomsky (1965).
 - **Performance is finite:** you can only utter/hear finite number of sentences in your lifetime.
 - **Competence is infinite:** you have the *ability* to utter/hear an infinite number of different sentences.
 - **Competence has a finite base.** Competence still has to be grounded in a finite enumeration of the rules of behavior.
- **Methodological breakthrough.** The distinction is essential to making progress on computational mechanism. It separates the underlying computational structure from how the system did on a particular occasion, with all its statistical noise.

- **Foundations of cognitive science.** The idea became foundational across the cognitive sciences. It is built into the idea that a machine could be computing a given function even if it fails to do so on particular occasion.

4 Finite state grammars

- **The grammar as machine.** Chomsky proposes that we view grammars as being generated by computational devices, starting with **finite state automata**.
 - “we can call the machine itself a finite state grammar” (p. 19)
- **Finite state grammar** passes through a series of states, each one of which produces a word, ending on a final state. The grammar is adequate if every sentence produced this way is grammatical.
- **The speaker as a machine.** “we can view the speaker as being essentially a machine of the type considered” (p. 20)
- **What it can do.** Chomsky makes it sound like it can’t do much. But you could get whole relative clauses, and it can be somewhat powerful if you make the loops a little bigger. See Figure 2.
- **But what are these grammars? The deep philosophical issue.** *Where or what* are these automata? They are not models of interpretive *process*, or the production *process*. They aren’t process models at all. So what could they be modeling that has cognitive reality?
 - Again, the analogy with the periodic table and the synthesis of elements.
 - An abstract methodology at the heart of much cognitive scientific practice.

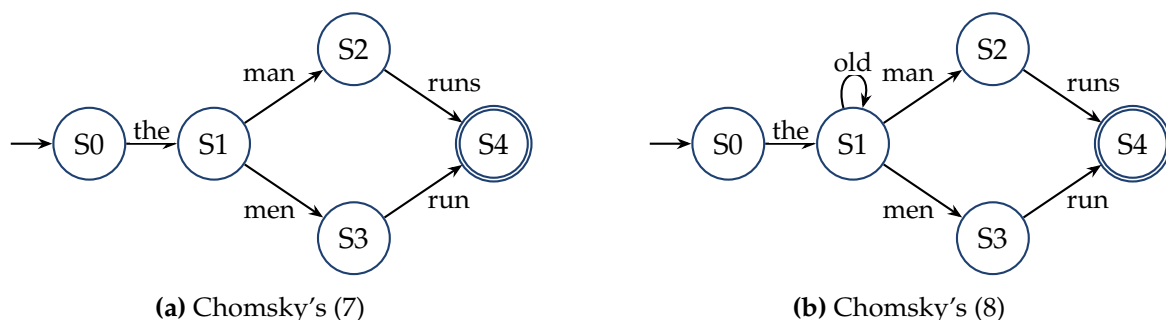


Figure 1. Finite state grammars for a fragment of English.

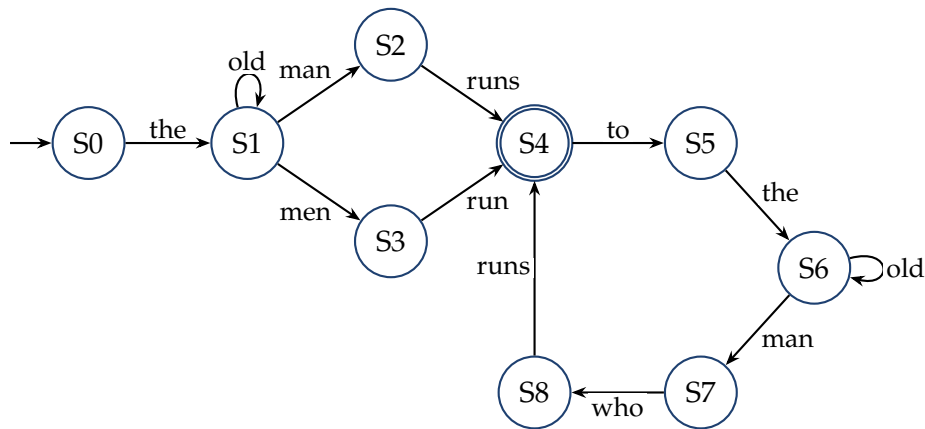


Figure 2. This larger loop captures the recursive production of relative clauses.

5 Limitations of finite state grammars

- **Chomsky's first negative theorem.**

(7) English is not a finite state language. (p. 21)

- **Can English be modeled by a finite state grammar?**

“it is impossible, not just difficult” (p. 21)

- **Consider three formal languages (p. 21):**

(8) (a) $ab, aabb, aaabbb, \dots$
($a^n b^n$)

(b) $aa, bb, abba, baab, aaaa, bbbb, aabbaa, abbbba, \dots$
(a string followed by its mirror image)

(c) $aa, bb, abab, baba, aaaa, bbbb, aabaab, abbabb, \dots$
(a string followed by an identical copy)

- None of these can be generated by a finite state grammar.
- **The English cases (p. 22).** Where $S_1, S_2, \dots$ are declarative sentences:

(9) (a) If S_1 , then S_2 .
(b) Either S_3 , or S_4 .
(c) The man who said that S_5 , is arriving today.

The problem of dependencies. Each of these cases involves a dependency between phrases, where the dependency spans the inserted sentence.

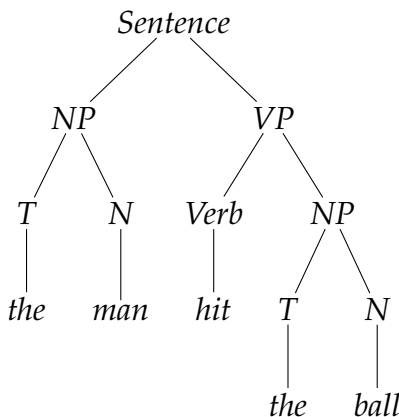
- The dependency-based sentences can recursively nest inside one another.
- **Compare:** parentheses in formal languages.
- **Why finite state grammars fail.** A finite state grammar is just a machine with finite states. To make sure that every *if* is followed by a *then*, the grammar must remember how many *ifs* are “open”. But there is no limit to how many *ifs* might be used, so there would have to be no limit on the size of the machines memory. Yet it only has finite memory. So it cannot handle unbounded nesting of clauses.
- **Why not impose a finite limit?** (pp. 23–24)
 - “Such arbitrary limitations serve no useful purpose, however.” (p. 23)
 - “a list is essentially a trivial finite state grammar” (p. 23)

6 Phrase structure grammars

- **Constituent analysis** is the familiar analysis of sentence into verbs, nouns, noun phrases etc. **Phrase structure grammars** formalize this idea.
 - **Rules.** $X \rightarrow Y$: “rewrite X as Y” (p. 26)
 - **Phrase structure grammar.** Here is a simple phrase structure grammar, where:
 - $NP = \textit{noun phrase}$
 - $VP = \textit{verb phrase}$
 - $N = \textit{noun}$
 - $V = \textit{verb}$
 - $T = \textit{determiner}$
- (10) (a) $\textit{Sentence} \rightarrow NP + VP$
 (b) $NP \rightarrow T + N$
 (c) $VP \rightarrow \textit{Verb} + NP$
 (d) $T \rightarrow \textit{the}$
 (e) $N \rightarrow \textit{man, ball, etc.}$
 (f) $\textit{Verb} \rightarrow \textit{hit, took, etc.}$
- **Derivation.** Here is a derivation of *the man hit the ball* (p. 27), using the rules above.

- (11) *Sentence*
- | | |
|-------------------------------------|-----|
| <i>NP + VP</i> | (a) |
| <i>T + N + VP</i> | (b) |
| <i>T + N + Verb + NP</i> | (c) |
| <i>the + N + Verb + NP</i> | (d) |
| <i>the + man + Verb + NP</i> | (e) |
| <i>the + man + hit + NP</i> | (f) |
| <i>the + man + hit + T + N</i> | (b) |
| <i>the + man + hit + the + N</i> | (d) |
| <i>the + man + hit + the + ball</i> | (e) |

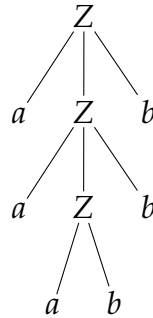
- **Parse tree** (p. 27). A parse tree is a diagrammatic representation of the history of derivation of the final string. Note the conceptual equivalence: **sentence structure** = **derivation history**.



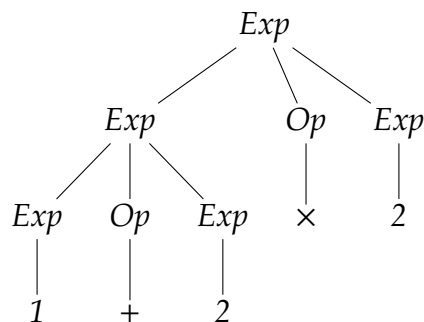
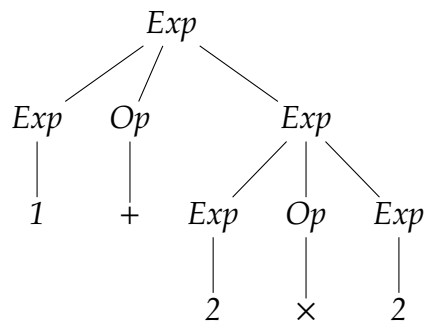
- **$[\Sigma, F]$ grammars** (pp. 29–30):
 - Σ : a finite set of initial strings; (in (10), just *Sentence*)
 - F : a finite set of transformational instructions $X \rightarrow Y$;
 - **Terminal string**: the last line of a derivation,
e.g. *the + man + hit + the + ball*
 - **Terminal language**: the set of terminal strings of some $[\Sigma, F]$ grammar.
- **Theorem** (p. 30). Finite state languages $\subsetneq$ terminal languages.
- **Grammar (12)** (p. 30):

- (12) Σ : Z
 F : $Z \rightarrow ab$
 $Z \rightarrow aZb$

- The tree for $aaabbb$ according to grammar (12).



- **Syntactic ambiguity as multiple derivations.** Consider the grammar
 - $Exp \rightarrow Exp\ Op\ Exp$;
 - $Exp \rightarrow 1\ |\ 2\ |\ 3\ |\ 4$;
 - $Op \rightarrow +\ |\ \times\ |\ \div$.
- In this grammar, the string $1 + 2 \times 2$ is ambiguous:



Memory and phrase structure

- **State memory.** Finite state grammar has only state memory.
- **Enduring memory.** Phrase structure grammar has some kind of enduring, arbitrarily long-term memory. This is what allows it to keep track of dependencies.

- **In the formal rules:** Each application of $Z \rightarrow aZb$ in (12) leaves a b to the right of Z . No matter how long Z is, it will always remember to produce a b if an a came earlier.
- **Pushdown automata.** It turns out (unknown to Chomsky at the time of *SS*) that a phrase structure grammar is equivalent to a pushdown automaton: a finite state machine plus a first-in, last-out stack memory.

7 Limitations of phrase structure grammars

- **Conjunction** (pp. 35–36).

- (13) (a) the scene — of the movie — was in Chicago
 (b) the scene — of the play — was in Chicago
- (14) the scene — of the movie and of the play — was in Chicago
- (15) (a) the — liner sailed down the — river
 (b) the — tugboat chugged up the — river
- (16) the — liner sailed down the and tugboat chugged up the — river
- **Chomsky's rule (26)** (p. 36). if $S_1 = \dots X \dots$ and $S_2 = \dots Y \dots$ are sentences that differ only in that X appears in one where Y appears in the other, and X and Y are constituents of the same type, then $S_3 = \dots X + \text{and} + Y \dots$ is a sentence. Formal statement, Appendix II, rule 22 (p. 113); X is “a minimal element (e.g., NP, VP, etc.)”

Structural analysis: of S_1 : $Z - X - W$

of S_2 : $Z - X - W$

Structural change: $(X_1 - X_2 - X_3; X_4 - X_5 - X_6) \rightarrow X_1 - X_2 + \text{and} + X_5 - X_3$

- **Why Chomsky's (26) isn't a phrase structure rule** (pp. 37–38).
 - “not only the final shape of these sentences, but also their ‘history of derivation’” (p. 37)
 - “a more powerful machine, which can ‘look back’ to earlier strings in the derivation” (p. 38)
 - “It makes essential reference to two distinct sentences” (p. 38)
- **A $[\Sigma, F]$ grammar as a machine** (p. 37):

$$\begin{aligned}
 (17) \quad & \Sigma: \text{Sentence} \\
 & F: X_1 \rightarrow Y_1 \\
 & \quad \vdots \\
 & X_n \rightarrow Y_n
 \end{aligned}$$

8 The Chomsky hierarchy and computing machines

- **Chomsky Hierarchy.** Grammars can be ranked by what kind of languages they produce *and* what kind of machines they correspond to.
- **Type 3: regular (finite state) grammars** $\leftrightarrow$ **finite state automata.** Memory: the current state only.
- **Type 2: context-free grammars** (phrase structure rules without context) $\leftrightarrow$ **pushdown automata.** Memory: one stack.
- **Type 1: context-sensitive grammars** (rules with context) $\leftrightarrow$ **linear bounded automata.** Memory: a tape no longer than the input.
- **Type 0: unrestricted grammars** $\leftrightarrow$ **Turing machines.** Memory: an unbounded tape. This is where the transformational grammars live.
- **Developed by Chomsky** in “Three models for the description of language” (1956) and “On certain formal properties of grammars” (1959).
- **English has a transformational grammar.**
- **Chomsky’s vision of linguistic shells:** the phrase structure grammar provides the “kernel” of English; transformational grammars extend it.

9 Grammars as theories

- **Theory.** “A grammar of the language L is essentially a theory of L.” (p. 49)
- **Corpus and grammar.** Chomsky speculates about what the relationship should be between corpus and grammar. He argues that, as theorists, we can use a corpus to get insight about our grammar. But we need not have an *algorithm* for getting from corpus to grammar.
 - “very questionable that this goal is attainable in any interesting way” (p. 52)

- “by intuition, guess-work, all sorts of partial methodological hints, reliance on past experience” (p. 56)

- **Two observations.**

- First, insofar as humans come to implement a grammar, even if they can’t explicitly represent it, it is corpus-based.
- Second, the same is true for modern AI models. We do think that constructing a machine that implements a grammar is possible given a corpus.

- **Implementing vs. representing a grammar.** To implement a grammar is not the same thing as explicitly representing it. We see the contrast in the relationship to developing a grammar from a corpus.

- **But what *are* these grammars? The deep philosophical issue.** *Where or what* are these automata? They are not models of interpretive *process*, or the production *process*. They aren’t process models at all. So what could they be modeling that has cognitive reality?

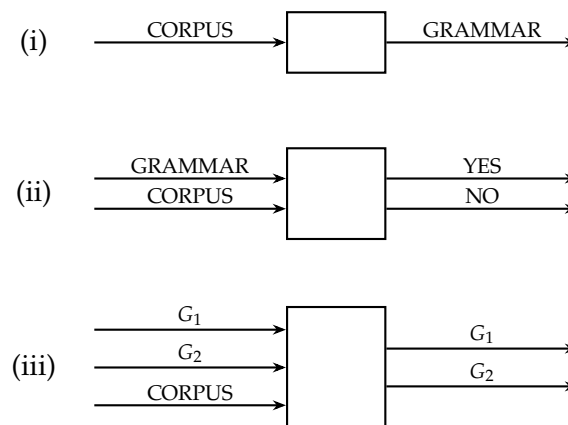


Figure 3. Chomsky’s (36) (p. 51): a theory that provides (i) a discovery procedure, (ii) a decision procedure, (iii) an evaluation procedure for grammars.

References

- Noam Chomsky. Three models for the description of language. *IRE Transactions on Information Theory*, 2(3):113–124, 1956.
- Noam Chomsky. *Syntactic Structures*. Mouton, The Hague, 1957.
- Noam Chomsky. On certain formal properties of grammars. *Information and Control*, 2(2):137–167, 1959.
- Noam Chomsky. *Aspects of the Theory of Syntax*. MIT Press, Cambridge, MA, 1965.